Bounded, Zero-Heap, Context-Aware Peripheral Recovery with Event Quarantine for Event-Driven Embedded Systems: A Host Evaluation and a Controlled I²C Hardware Demonstration

Sujit Sai Y[a], Mithul Rashwanth SR[a], Rohit Mathur[b,*]

[a] *School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore 632014, India*

[b] *School of Electronics Engineering (SENSE), Vellore Institute of Technology (VIT), Vellore 632014, India*

* Corresponding author. E-mail: rohit.mathur@vit.ac.in

ABSTRACT

Peripheral faults in firmware are usually handled by fixed retry loops, or by recovery machinery that needs checkpoints, logs or an operating system. We ask how far a small, statically sized runtime can go. We implemented a zero-heap C runtime with a bounded event queue, exact-event quarantine, one recovery episode and five in-house policy variants (P0–P4). P4 combines fault-to-event association precision, a short attempt history and safety flags in a finite action ladder. In a frozen deterministic host campaign (16 case families, 5 policies, 5 repetitions, 400 executions), P4 closed the recovery episode in all 16 families; P0 and P1 left it active in two and P2 and P3 in four. The differences are confined to three families, appear partly driven by a runtime defect that leaves an episode open on an invalid target, and are offset by one family where a simpler policy needed one attempt against three. A corrected comparison is not established. On an STM32 NUCLEO-L476RG with a BME280, a controlled I²C address NACK was followed by verified sensor readback in one policy comparison (P1 and P4 passed), one P4 run and five control trials. No stuck-bus, timing, memory or general superiority claim is made.

Keywords: Peripheral fault recovery; Event quarantine; Zero-heap runtime; Embedded systems; I²C

1. Introduction

Microcontroller firmware talks to sensors, flash and radios through buses such as I²C and SPI. These peripherals fail in ways the application cannot prevent: a device returns a negative acknowledgement (NACK), a transfer times out, arbitration is lost or a bus error is flagged. The common response is a retry loop with a fixed bound. It is simple and predictable, but it treats every fault alike. It does not ask whether a retry is safe for the pending transaction, whether the fault can be tied to one event, or whether other queued work depends on the failed one. Embedded dependability surveys organize the wider space of software and hardware mitigation (Solouki et al., 2024; Safari et al., 2022), and software fault-tolerance work describes recovery by retry, reboot and failover (Trivedi et al., 2024).

Stronger mechanisms exist for peripheral and driver recovery. Phoenix logs peripheral accesses, tracks program and external peripheral state, and rolls back to the failed access (Smith & Rixner, 2015). Shadow drivers take over the role of a failing driver while the system keeps running (Swift et al., 2006). These mechanisms carry costs that small event-driven firmware may not accept, such as checkpoint memory, replay logs or an operating-system layer.

This paper asks a narrower question: *how far can a small, statically sized runtime go in recovering from peripheral faults by using only the evidence at hand?* That evidence is the fault class, how precisely the fault can be tied to one event, a short attempt count and a few safety flags. We built a zero-heap C reference runtime that holds all state in one fixed structure, implemented five policy variants in it, compared them in a deterministic host campaign, and exercised the decision path on a real I²C sensor.

Scope note. The host comparison is read as follows. The baselines P0–P3 are our own ablations, not implementations of published algorithms, and exact-event quarantine is shared by all five variants. A runtime defect that leaves a recovery episode open after an action on an invalid target partly explains the P4 differences in two families (CF04 and CF07). The frozen campaign does not show whether those differences would survive on a corrected runtime, so they must not be read as general superiority.

We address four research questions.

- **RQ1.** How does the implemented runtime make recovery decisions, and what bounds its state?

- **RQ2.** How do the in-house policies P0–P4 differ under matched deterministic scenarios, and which differences can be attributed to P4's own rules?
- **RQ3.** After a controlled fault on a real I²C peripheral, does the policy-selected recovery path restore a fresh, verified sensor read?
- **RQ4.** What can be concluded about the relationship between the software reference runtime and the physical validation?

The paper makes five contributions, classified by type in Table 2 (Section 3): a bounded zero-heap runtime, a five-policy comparison framework, a bounded set of attributable empirical differences, a narrow physical validation, and an explicit separation of software-model and physical evidence. We do not claim that P4 is superior in general, that any single mechanism is new, or that the combination is unprecedented. Section 2 reviews related work. Sections 3 to 5 define the problem, the design and the implementation. Sections 6 and 7 describe the experiments and report results. Sections 8 to 10 discuss, qualify and conclude.

2. Background and Related Work

We reviewed a focused set of sources on peripheral recovery, driver and component recovery, peripheral-interface robustness, I²C fault handling and dependability. The review is not systematic, and Table 1 reports only what the cited abstracts and records state; cells marked "not assessed" were not checked in full text.

2.1. Peripheral, driver and component recovery

Phoenix targets resource-constrained embedded systems, tracks peripheral state and rolls back to the failed access, and reports needing as little as 6 KB of memory (Smith & Rixner, 2015). KARMA addresses a different fault source: intermittent power makes asynchronous peripheral operations inconsistent, and it provides a general abstraction for them (Branco et al., 2019). RESTOP is middleware that retains external peripheral state, for SPI and I²C devices, across power interruptions (Rodriguez Arreola et al., 2018). Shadow drivers (Swift et al., 2006) and microreboots (Candea et al., 2004) recover whole drivers or fine-grained components in operating systems and server software. SeRoHAL generates hardware abstraction layers with software safety mechanisms for errors in peripheral interfaces, and selects protection by the criticality of each access (Kleeberger et al., 2019). Experimental work on I²C faults in nanosatellites catalogs bus fault conditions and mitigation strategies (Albalooshi et al., 2023).

2.2. Dependability and runtime monitoring

We use the fault, error and failure terminology of the dependability taxonomy (Avizienis et al., 2004). Runtime verification offers a complementary view. Resource-bounded monitoring for embedded real-time systems (Reinbacher et al., 2014), online monitoring for fault disambiguation on a robot (Kempa et al., 2020) and monitoring guided by ISO 26262 (Heffernan et al., 2014) detect and classify faults, whereas our runtime decides and acts. Transient-fault tolerance by task-level redundancy (Asghari et al., 2018), data-error detection and recovery (Thati et al., 2017) and fault tolerance under power and real-time constraints (Santos et al., 2009) frame the problem but do not describe an event-level peripheral recovery policy.

2.3. Fault-injection methodology

Fault injection is the standard way to evaluate recovery mechanisms experimentally. Software-implemented injection can use debugging and performance-monitoring hardware of a processor to inject faults and observe their activation (Carreira et al., 1998), and surveys describe how such techniques are applied to fault-tolerant systems and how they trade fault representativeness against efficiency and usability (Natella et al., 2016). A comparison of physical and software-implemented techniques on one fault-tolerant system found that the technique used can change the observed impact of an injected fault (Arlat et al., 2003). We use these works only for methodology and for the validity discussion: our address NACK is a software-driven bus condition, so it stands in for a physical fault and does not represent every physical failure mode.

Table 1. *Literature comparison. Cells state what each source reports in the text we inspected; "not assessed" means not checked in full text.*

Study	Target	Fault model	Recovery mechanism	Evaluation	Difference from this study
Smith & Rixner (2015)	Constrained embedded systems	Peripheral failures	Access logging, state tracking, rollback	Memory as low as 6 KB reported	Needs checkpoint and replay; we use bounded decisions with no replay log

Study	Target	Fault model	Recovery mechanism	Evaluation	Difference from this study
Branco et al. (2019)	Intermittently powered devices	Power-induced inconsistency	Abstraction for async peripheral operations	Not assessed	Different fault source (power), not event association
Rodriguez Arreola et al. (2018)	MCU middleware, SPI and I²C	Power interruption	Retain and restore peripheral state	Empirical evaluation reported	State restoration; no fault-to-event policy
Swift et al. (2006)	OS device drivers	Driver failure	Shadow driver takes over during recovery	Not assessed	Operating-system setting
Candea et al. (2004)	Server software components	Component failure	Fine-grained restart	Not assessed	Component restart; not firmware peripherals
Kleeberger et al. (2019)	Mixed-criticality software	Hardware errors in peripheral interfaces	Generated HAL protection by criticality	Robot-arm software	Protects register accesses; no event quarantine or episode ladder
Albalooshi et al. (2023)	Nanosatellite I²C buses	I²C bus faults	Mitigation strategies analysed	Experimental analysis	Protocol-level analysis; not a runtime policy
This study	MCU event runtime, zero heap	Address NACK on hardware; 16 injected families on host	Association-aware ladder with quarantine	Host: 400 runs; hardware: one controlled NACK setup	In-house baselines only; narrow hardware scope

2.4. Research gap and scope of the claim

Among the sources reviewed, embedded peripheral recovery relies on checkpointing or state restoration (Smith & Rixner, 2015; Branco et al., 2019; Rodriguez Arreola et al., 2018), and driver or component recovery targets operating systems or servers (Swift et al., 2006; Candea et al., 2004). Selective protection of peripheral accesses exists (Kleeberger et al., 2019). We did not find, in this set, a policy that limits its action to the precision of the fault-to-event association, runs in a statically sized runtime and quarantines only the associated event. **This statement is limited to the sources reviewed.** The project's novelty audit classifies the combination as an unresolved hypothesis, and it records that individual mechanisms (retry, reinitialization, bounded state, escalation) are established. We therefore claim the implementation and evaluation of one integrated mechanism, not a new mechanism and not a legal novelty or patentability finding.

3. Research Problem, Questions and Objectives

Problem. A fixed retry loop cannot tell a transient fault from a persistent one, cannot tell whether the fault belongs to one event, and cannot stop other work from being blocked. Recovery machinery from larger systems may not fit a small runtime. The problem is to choose a recovery action from limited evidence, with bounded memory, bounded attempts and a verified outcome. The objectives are to implement such a runtime (RQ1), compare policies under identical conditions (RQ2), validate the decision path on a real peripheral (RQ3) and keep the two kinds of evidence separate (RQ4). Table 2 classifies each contribution by type and by the strength of its support.

Table 2. *Contribution classification.*

Contribution	Type	Supporting evidence	Boundary
Statically bounded C runtime (no heap calls)	Implementation	Source search, fixed capacities, tests passing	No measured MCU timing, stack or energy
Five-policy framework P0–P4 in one runtime	Evaluation methodology	Same events and schedules for all policies	Baselines are in-house; quarantine shared by all
Differences in CF04, CF05, CF07	Empirical finding	Frozen 400-run campaign	Tied to an episode-leak path; not general superiority
Controlled NACK recovery on STM32L476RG/BME280	Experimental	Debugger-visible variables	One setup; not a stuck bus; one run per policy
Software/hardware evidence separation	Methodology	Distinct paths, tables and captions	No live UI-to-board link
Integration of the mechanisms as a whole	Possible novelty	Narrow search only	Unresolved; not claimed
Stuck-bus tests, MCU profiling, independent baselines	Future work	None	Not done

4. System Architecture and Design

Fig. 1 shows the two evidence paths. They share the runtime source but are separate programs: the software path runs the runtime on a host behind a browser interface, and the physical path links the same runtime source into STM32 firmware observed through a debugger. Fig. 1 is an explanatory diagram drawn from the repository, not an experimental observation.

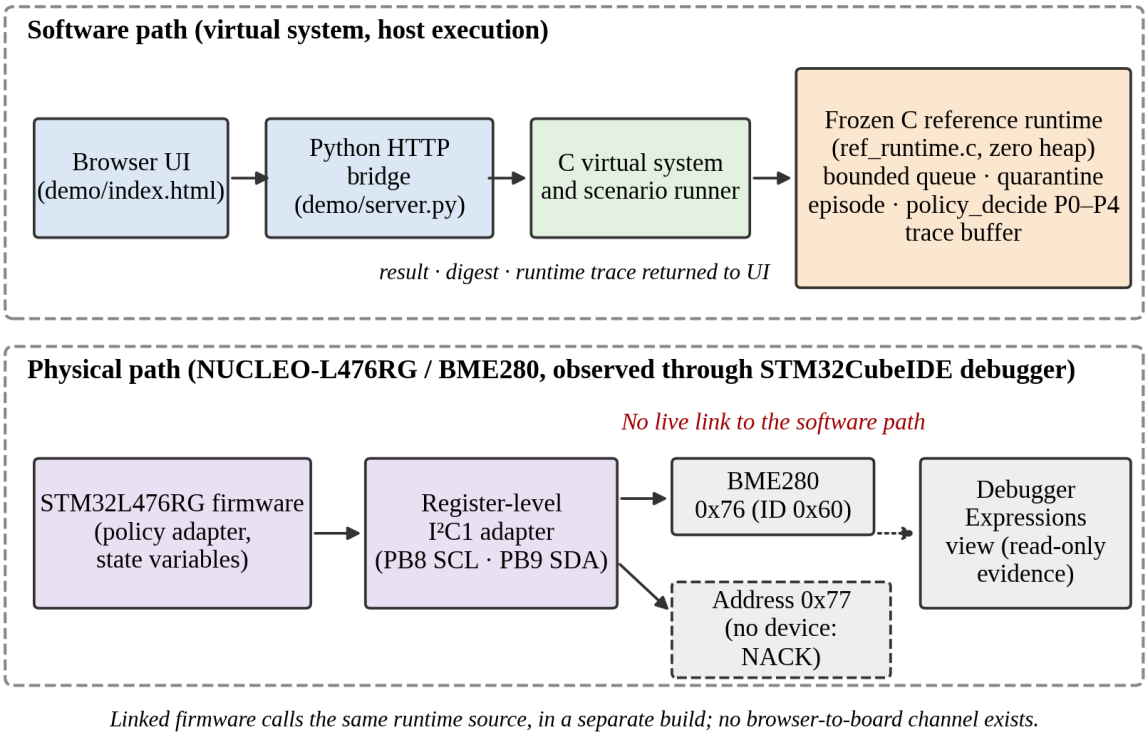


Fig. 1. *Software path (top) and physical path (bottom). The firmware shares the runtime source with the virtual system but has no communication channel to the browser interface.*

4.1. Runtime components and state

The runtime targets single-threaded, event-driven firmware. An event is a peripheral read, write, service, housekeeping or recovery-control operation (`EventType`) with a criticality and a dependency list. All state lives in one `Runtime` structure with static capacities: `QMAX = 16` queued events, `XMAX = 4` quarantined events, `ACTIVE_MAX = 20` slots, `DMAX = 4` dependencies per event, `EPMAX = 1` recovery episode and `TRACE_EVENT_CAP = 256` trace records. A reference to an event carries a slot and a generation, so a stale reference to a recycled slot is rejected (`OUTCOME_STALE_REF`).

4.2. Event and fault path

`runtime_admit` places an event in the bounded queue, and the scheduler dispatches the first event whose dependencies allow it. When an execution fails, `runtime_start_fault` opens the single recovery episode with a fault class, an association level (exact event, service only, peripheral only or unknown), the safety flags and the attempt count. These inputs are supplied by the caller; the runtime does not infer them. If the association is exact, the failed event moves to quarantine and leaves the queue while other events continue. `policy_decide` returns one of five actions: none, retry, reinitialize or reset, degrade, escalate. After a verified retry or reinitialization, `runtime_verify_and_release` returns the event to the queue and closes the episode. A terminal degrade or escalate closes the episode and marks the event discarded or escalated.

4.3. Policy variants

All five variants are branches of `policy_decide` in one source file, so they differ only in the decision rule (Table 3). Fig. 2 draws the P4 ladder from the source. P4 first requires exact association, then checks dependency and shared-state safety, then applies a hard limit of three attempts, and otherwise chooses retry, reinitialization or degradation by the safety flags.

Table 3. *Decision rules of the five variants, in priority order (source: `policy_decide`). Attempt is the number of recovery actions already taken in the episode.*

Variant	Rule
P0 fixed retry	Retry while attempt < 2, otherwise escalate. Ignores all context.
P1 retry + recovery	Retry if attempt < 2 and retry is safe; else reinitialize if safe and attempt < 3; else degrade if safe; else escalate.
P2 context only	Degrade or escalate if association is unknown, dependencies are unsafe or shared state is invalid. Else reinitialize for a persistent fault if safe; else retry if safe; else degrade or escalate. No attempt limit.
P3 context + history	As P2, but reinitialize once attempt ≥ 2 if safe, and retry only while attempt < 2. No attempt limit.
P4 integrated	Escalate unless association is exact; degrade or escalate if dependencies are unsafe or state is invalid; terminate at attempt ≥ 3; at attempt 2 reinitialize if safe; otherwise retry if safe, else reinitialize if safe, else degrade, else escalate.

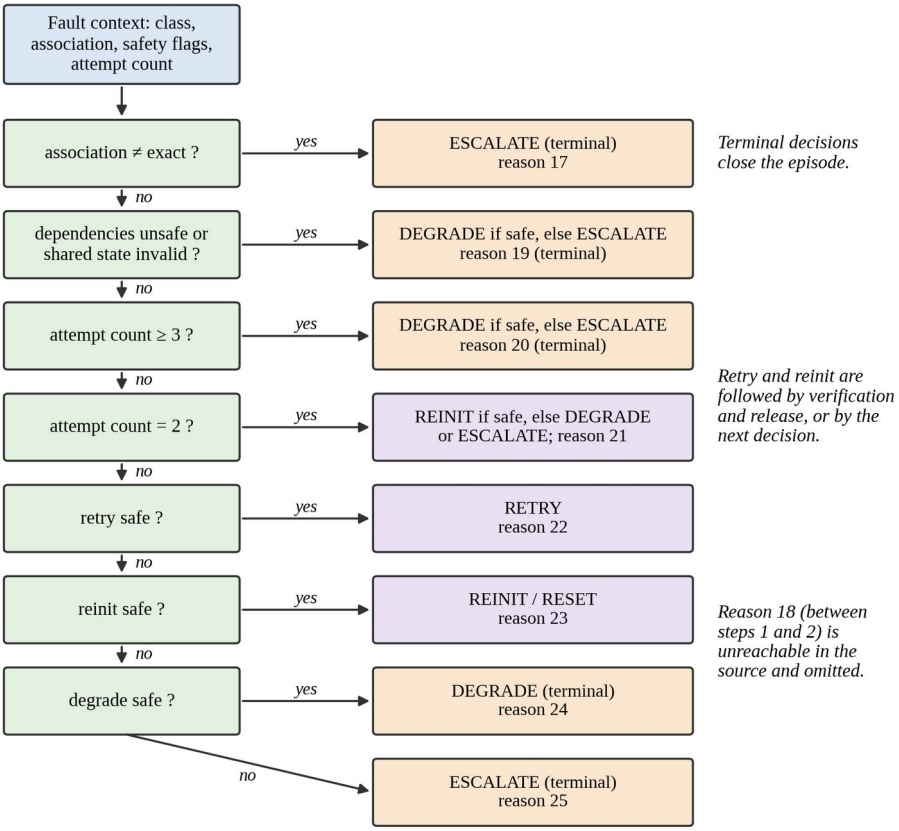


Fig. 2. *P4 decision ladder drawn from `policy_decide`. Reason 18 is unreachable in the source and omitted. This is an explanatory figure.*

4.4. Quarantine and episode semantics

Quarantine of the exact event is applied by `runtime_start_fault` for every variant. It is therefore a shared mechanism and not a P4-only feature, and the host campaign confirms identical work preservation across P0–P4 in the queue-pressure and high-load families. The runtime keeps one episode at a time. If an action leaves the episode open, later faults are rejected with `OUTCOME_EPISODE_FULL`, and this is the mechanism behind two of the P4 differences reported in Section 7.

4.5. Bounded state and the zero-heap claim

Table 4 lists the evidence for each boundedness claim. The zero-heap claim rests on a source-level search and fixed storage, not on a runtime measurement. The bounds are operation counts, not times.

Table 4. *Evidence for boundedness claims.*

Claim	Evidence	What it does not show
No dynamic allocation	Text search of the runtime source and header finds no `malloc`, `calloc`, `realloc` or `free`	No link-map or runtime allocation check on the MCU

Claim	Evidence	What it does not show
Fixed state size	Host `sizeof`: Runtime 8164 B, EventSlot 84 B, trace 6144 B	Not MCU Flash or RAM
Bounded decision cost	`policy_decide` has no loop; other loops run over at most ACTIVE_MAX or QMAX entries	No execution time measured
Finite recovery ladder	P0, P1 and P4 stop after 2, 3 and 3 actions; P2 and P3 have no attempt limit	Attempts are not time
Build size (hardware)	Linker output text 12,948 B, data 112 B, BSS 12,704 B, total 25,764 B	Not stack use or the runtime's own RAM cost; binary identity unverified

Two source-level findings qualify the bounds. Both were reproduced in a scratch build outside the repository. First, a quarantined event leaves the queue, so other events can fill all 16 positions; releasing the event then calls `queue_append` with no capacity check, which aborts with assertions on and writes one element past the queue array with `NDEBUG`. Second, each generation counter is 8 bits with 0 reserved, so after 255 admissions through one slot `runtime_admit` returns `OUTCOME_STALE_REF` while the queue is empty. The trace buffer also stops silently at 256 records. A separate candidate, `reference_v2`, addresses these paths. Its campaign outputs, source version and hashes are not verified for this paper, so it is neither reported nor mixed with the frozen campaign.

5. Implementation

5.1. Reference runtime and host tools

The runtime is written in C11 (`reference/src/ref_runtime.c`, `reference/include/ref_runtime.h`). The project's documented build uses CMake with MSVC on Windows, and the authors report that the test suite passes there. The host harness (`reference/experiments/`) reads declarative schedules (`workloads.csv`, `fault_schedules.csv`) and writes one record per run. The virtual system (`reference/virtual/`) wraps the runtime with modelled peripherals and scenario definitions.

5.2. Firmware and physical setup

The target is an ST NUCLEO-L476RG (STM32L476RG) with a BME280 breakout. The sensor is at I²C address `0x76` (chip-ID register `0xD0`, expected value `0x60`), wired VIN to 3V3, GND to GND, SCL to PB8 (D15) and SDA to PB9 (D14). A register-level I²C1 adapter (timing value `0x30420F13`) is built together with the unchanged runtime source in STM32CubeIDE 2.2.0 and flashed through ST-LINK. The controlled fault reads the chip-ID register at address `0x77`, where no device answers, which yields an address NACK (`E_NACK = -1`). Results are exposed as debugger-visible variables and a terminal breakpoint, so no console output is needed. Fig. 4 shows the setup.

Which runtime calls the firmware makes. We inspected the committed sources. The P4 single-run firmware (`main.c`) calls `runtime_init`, `runtime_admit`, `runtime_start_fault`, `policy_decide` and `runtime_verify_and_release`, and it does not call `runtime_apply_policy`; the firmware itself performs the I²C reinitialization. This matches its README. The five-policy firmware exists in three committed variants. In each, retry and reinitialization are executed by the firmware, and `runtime_apply_policy(&rt, 0)` is called only in the terminal branch, to apply the terminal decision inside the runtime; the diagnostic and formatted variants call it at lines 488 and 444, and the original variant at line 133. The README statement that the integration does not call it describes `main.c` only. The authors identified a source file, `hardware_exp.c` (SHA-256 `8d65294a7dbf8aca22cab9f2c56732bdb015bf7c3fab87cee6b555f7ae6a1ee8`), as the program used for the policy comparison. The hash identifies the source bytes only. It is not among the committed variants, and the exact flashed binary has not been hash-linked to any source, so binary identity is unverified and which variant ran remains unresolved.

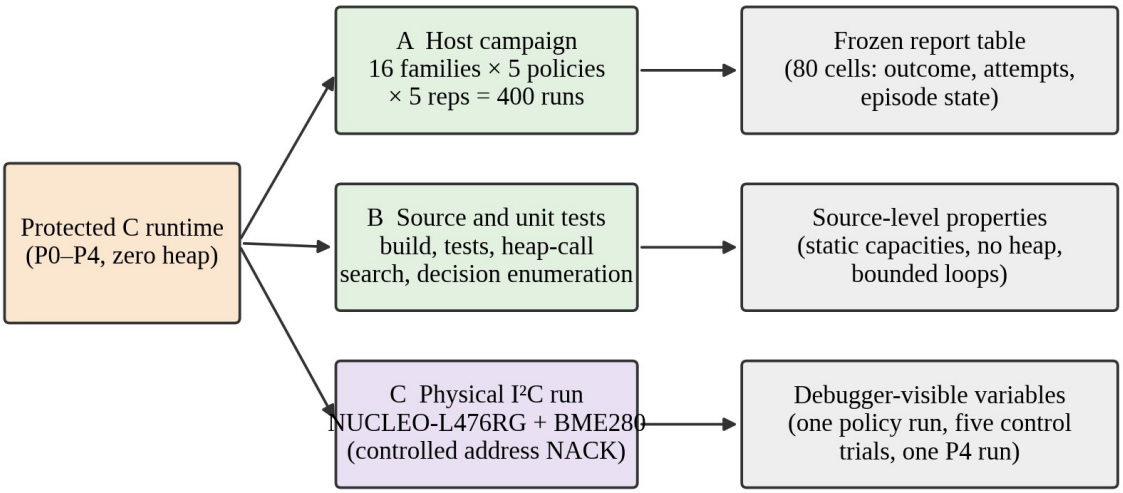
5.3. Virtual-system demonstrator

A browser interface (`demo/index.html`) sends requests to a Python HTTP bridge (`demo/server.py`), which calls a compiled C virtual-system program linked with the reference runtime. The runtime remains the authority for decisions, and the interface shows the outcome, a digest and correlated runtime and wrapper traces. Disconnecting a virtual peripheral makes admission fail visibly. The demonstrator is software only. Its

interface also shows a panel for a proposed future target mapping to a NUCLEO-U575ZI-Q board with an STM32U575ZIT6Q; that panel is a plan and is not the physical experimental board.

6. Experimental Methodology

Three evidence streams are kept apart (Fig. 3, Table 5). Their results are never pooled into one sample.



Streams A, B and C are reported separately and never pooled into one sample.

Fig. 3. *Evidence streams. A: frozen host campaign. B: source-level checks and tests. C: physical I²C observations. This is an explanatory figure.*

Table 5. *Evidence streams and their scope.*

Stream	What was run	Counts	Cannot support
A. Host campaign	16 families × 5 policies × 5 repetitions in the C harness	400 executions, 80 cells	Physical behaviour; timing; independent baselines
B. Source and tests	Heap-call search, decision enumeration, unit and virtual tests	591,360 decision contexts enumerated	Runtime behaviour on the MCU
C1. Policy comparison (hardware)	One pass over P0–P4 with an address NACK	1 run per policy	Repeatability; statistics
C2. Control trials (hardware)	NACK then read, without full I²C initialization between	5 trials in one firmware execution	Need for reinitialization in general
C3. P4 single run (hardware)	Baseline, NACK, P4 reinit, fresh read, terminal report	1 run	Repeatability
D. Demonstrator	Browser to bridge to virtual C runtime	Not an experiment	Any hardware claim

6.1. Software and deterministic policy evaluation

The campaign crosses 16 case families (CF01–CF16), five policy variants and five repetitions, for 400 executions with identical inputs per policy. The families cover persistent faults with history, queue pressure, dependency order, association ambiguity, a fault during recovery, recovery failure, a subsequent fault after an ambiguous episode, critical versus deferable work, repeated contention, follow-up work after reinitialization, alternating fault classes, queue position, low-load and high-load controls, a negative control and a baseline-lower-cost case. Each cell is reported as a terminal outcome and an attempt count: success, terminal escalate, terminal degrade, stale reference, no terminal outcome within the harness bound, or episode full. A flag records whether the episode was left active. A difference is credited to P4 only after checking whether a shared mechanism explains it. The runtime source and header were unchanged from the campaign's start commit; their current SHA-256 values are in Appendix A. The original per-repetition output files (`k10_runs.csv` and `k10_traces.csv`) were not found in the repository and were not reconstructed under those names, so we report the frozen table and could not re-analyse repetition-level data independently.

6.2. Physical hardware validation

The hardware procedure is: (1) condition the bus and read the chip ID at `0x76` as a baseline; (2) read at `0x77` and require a NACK; (3) open an episode with a fixed context (I²C NACK, exact association, retry not safe, reinitialization safe) and call `policy_decide`; (4) if the policy selects reinitialization, reinitialize I²C1; (5) read the chip ID at `0x76` again; (6) require `0x60` before `runtime_verify_and_release`; (7) record the terminal state at a breakpoint. The diagnostic firmware also evaluates `policy_decide` for 80 further contexts (5 policies × 16) without bus access, and these are decisions, not physical fault injections. The fault is an address NACK. It is not a physically stuck SDA or SCL line.

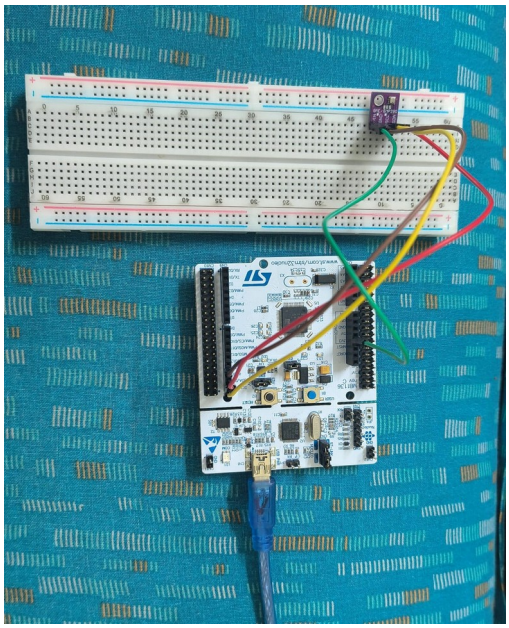


Fig. 4. *Physical setup: NUCLEO-L476RG, BME280 breakout, breadboard, jumper wires and USB connection. The photograph documents the arrangement only; outcomes are shown in Fig. 6 and Tables 7 to 9.*

6.3. Software demonstrator

The demonstrator is treated as a visualization and modelling tool. Its role is to display scenarios, policy selection, frozen evidence, runtime status and traces. It is not counted as a hardware experiment, and the browser never controlled the board.

6.4. Reproducibility

Requirements: Git, a C11 compiler, Python 3, and for the documented route CMake 3.16 or newer. The authors' route (Windows PowerShell, MSVC) is `cmake -S reference -B reference/build`, `cmake --build reference/build --config Debug` and `ctest --test-dir reference/build -C Debug --output-on-failure`. With the non-MSVC flags in the CMake file (`-Wall -Wextra -Werror -pedantic`), GCC 13.3 rejects the runtime with `-Wmisleading-indentation` errors. We therefore also built the tests directly with GCC 13.3 on Ubuntu 24.04, with that one warning disabled:

```
gcc -std=c11 -Wall -Wextra -pedantic -Wno-misleading-indentation -Ireference/include \
    reference/src/ref_runtime.c reference/tests/test_reference.c -o reference_tests &&
./reference_tests
```

The three suites (`reference_tests`, `virtual_system_tests`, `virtual_baseline_tests`) passed in this build at repository commit `982dad1`. To regenerate the host campaign, work in a scratch copy and never in the frozen output location: build `reference/experiments/k10_experiment.c` with the runtime into `reference/build/k10_experiment`, then run `python3 reference/experiments/evaluate_k10_advantage.py --report <path>` with a real output path. This printed "executed 400 matched runs" and reproduced all 80 cells of the frozen table, with every repetition identical. The regenerated files serve only as a consistency check. They are not the original raw outputs, which were unavailable for independent re-analysis, and they are not used as data of record. The hardware experiment has no one-command path: it requires the board, STM32CubeIDE, the firmware in `Plan2.0/firmware/bme280_policy_integration/` and a debugger session in which the variables of

Tables 7 to 9 are added as expressions. Compiler, HAL and ST-LINK versions beyond STM32CubeIDE 2.2.0 were not recorded.

7. Results

7.1. Host campaign (stream A)

Fig. 5 shows every cell and Table 6 summarizes outcomes per policy over the 16 families (one result per family, since repetitions were identical).

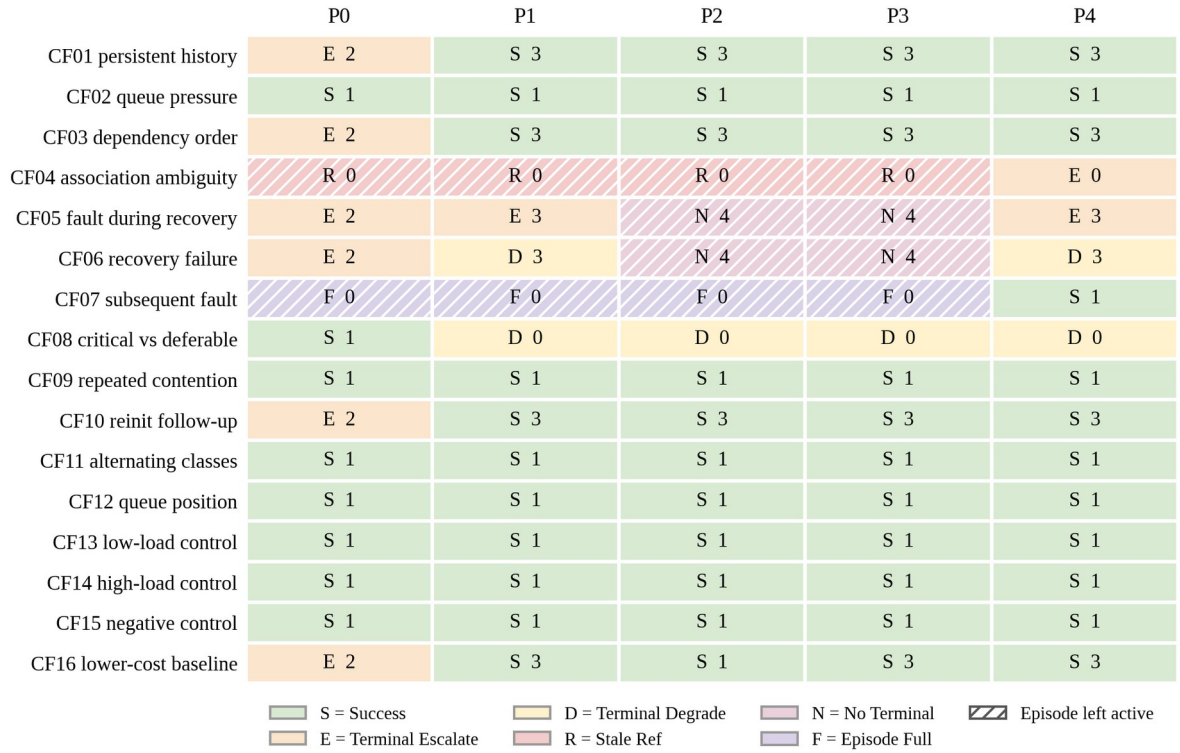


Fig. 5. *Outcome and attempt count for each family and policy in the frozen host campaign. Hatching marks an episode left active. The letters are defined in the legend.*

Table 6. *Outcomes over 16 families per policy. Success counts are not a ranking.*

Policy	Success	Term. escalate	Term. degrade	Stale ref	No terminal	Episode full	Episode left active
P0	8	6	0	1	0	1	2
P1	11	1	2	1	0	1	2
P2	11	0	1	1	2	1	4
P3	11	0	1	1	2	1	4
P4	12	2	2	0	0	0	0

Where P4 differs. In CF04 (non-exact association), P0–P3 return a stale-reference outcome and leave the episode active, whereas P4 escalates and closes it. In CF07 (a second fault after an ambiguous one), the unclosed episode blocks the second fault in P0–P3, whereas P4 recovers in one attempt. In CF05 and CF06 (faults during recovery), P4 differs only from P2 and P3, which stop at four attempts with no terminal outcome; P0 and P1 also terminate. By the source, CF04 and CF07 arise because P0–P3 select a retry for a target that is not a valid event reference, and `runtime_apply_policy` then returns `OUTCOME_STALE_REF` without closing the episode. We therefore disclose that this defect appears to contribute to the CF04 and CF07 advantages. The frozen campaign cannot show whether P4 would keep them against baselines on a corrected runtime, and we do not claim that it would.

Where P4 does not help. Seven families (CF02, CF09, CF11, CF12, CF13, CF14, CF15) give identical results for all five policies. Persistent-fault recovery (CF01, CF03, CF10) succeeds in P1–P4 through reinitialization, which P1 already has. In CF16, P2 reinitializes at once and succeeds in one attempt, whereas P4 retries twice

first and needs three. Attempt counts are not times. In CF08, P0 completes the work while P1–P4 degrade deferable work, and whether degrading is the intended behaviour is not stated.

7.2. Physical observations (stream C)

Values were read from debugger variables. Table 7 lists the P4 single run, Table 8 the five-policy comparison and Table 9 the five control trials. Fig. 6 shows the debugger captures for the comparison.

Table 7. *P4-integrated single run (stream C3), as recorded in the project's hardware records.*

Variable	Value	Meaning
`baseline_ok`	1	Initial BME280 read succeeded
`fault_attempted`, `fault_nack_observed`	1, 1	Controlled NACK attempted and observed
`recovery_attempted`	1	Recovery path attempted
`post_recovery_read_ok`	1	Fresh read succeeded
`post_recovery_chip_id`	96 (0x60)	Expected BME280 chip ID
`sensor_error`	0	No error at terminal result
`test_state`, `test_finished`	3, 1	Recovery passed; terminal report reached
`test_report_code`	8	Success report

The earlier firmware-only test, before runtime integration, additionally recorded `fault_result = -1` and `recovery_error = 0`. The policy and runtime auxiliary fields of the P4 run (decision action and reason, release result) were not captured, so they are not reported as measured. No original image of this readout is available, so the values come from the project's hardware record and are not shown as a screenshot.

Table 8. *Five-policy comparison, one run per policy (stream C1).*

Quantity	P0	P1	P2	P3	P4
Wrong-address read result (`E_NACK`)	−1	−1	−1	−1	−1
Physical status (7 terminal, 1 pass)	7	1	7	7	1
Runtime result (4 = terminal escalate)	4	1	4	4	1
Release result (1 = success)	0	1	0	0	1
Post-recovery chip ID	0	96	0	0	96

The harness reached its own end state (`progress_stage = 900`, `finished = 1`, `overall_code = 1`), which does not mean all policies recovered. P1 and P4 reinitialized, read `0x60` and released the event. P0, P2 and P3 ended in terminal escalation, which is a policy outcome and not a processor crash. For the fixed context, the host-computed first decisions match the observed pattern: reinitialize for P1 and P4, retry then escalate for P0, and escalate for P2 and P3. The outcome therefore follows from the rules and from the flags we set (retry not safe, reinitialization safe). It does not show that the policies chose well for the physical fault. This is one run per policy and no variance exists.

(a)

physical_status[0]	volatile uint32_t	7
physical_status[2]	volatile uint32_t	7
physical_status[1]	volatile uint32_t	1
physical_status[3]	volatile uint32_t	7
physical_status[4]	volatile uint32_t	1

(b)

release_result[1]	volatile uint32_t	1
release_result[2]	volatile uint32_t	0
release_result[3]	volatile uint32_t	0
release_result[4]	volatile uint32_t	1
post_read_ok[1]	volatile uint32_t	1
post_read_ok[4]	volatile uint32_t	1
post_chip_id[1]	volatile uint32_t	96
post_chip_id[4]	volatile uint32_t	96
release_result[1]	volatile uint32_t	1
release_result[4]	volatile uint32_t	1
finished	volatile uint32_t	1
overall_code	volatile uint32_t	1
Add new expression		

(c)

post_chip_id[4]	volatile uint32_t	96
nack_result[4]	volatile int32_t	-1
nack_result[3]	volatile int32_t	-1
nack_result[2]	volatile int32_t	-1
nack_result[1]	volatile int32_t	-1
nack_result[0]	volatile int32_t	-1
release_result[0]	volatile uint32_t	0
release_result[1]	volatile uint32_t	1
release_result[2]	volatile uint32_t	0
release_result[3]	volatile uint32_t	0
release_result[4]	volatile uint32_t	1
Add new expression		

(d)

progress_stage	volatile uint32_t	900
progress_policy	volatile uint32_t	4
progress_scenario	volatile uint32_t	15
progress_attempt	volatile uint32_t	0
diag_phase	volatile uint32_t	0
finished	volatile uint32_t	1
overall_code	volatile uint32_t	1
Add new expression		

Fig. 6. *Debugger captures from the policy-comparison run, stacked without changing values. (a)* `physical_status`*. (b) Post-read, chip ID and release values. (c) NACK and release results. (d) Harness terminal state. They show one run and no stuck-bus condition.*

Table 9. *Control trials (stream C2): NACK at* `0x77` *followed by a read at* `0x76` *without a full* `iinit()` *call in between.*

Trial	NACK result	NACK observed	Post-NACK read OK	Chip ID
0–4 (each)	−1	1	1	96 (0x60)

All five trials observed the NACK and then read the expected chip ID. For this absent-address NACK, a full peripheral initialization was therefore not needed to obtain the next read. The transaction helper may still perform local abort and flag handling, so this does not isolate every cleanup step. It also means the hardware run does not show that reinitialization was necessary here. The trials are five repetitions inside one firmware execution.

7.3. Resource figures

The build console reported 0 errors and 0 warnings and linker sizes of 12,948 B text, 112 B data and 12,704 B BSS (25,764 B total). These are build figures and not a runtime stack, timing, energy or complete resource profile.

7.4. Demonstrator

Fig. 7 shows the demonstrator at its initial step: scenario and policy selection, a virtual hardware workbench, a frozen-evidence panel, runtime status and empty trace panels. It documents the software model. Its "future target mapping" panel names a proposed NUCLEO-U575ZI-Q / STM32U575ZIT6Q target. That is a proposed mapping, not the physical board, which was the NUCLEO-L476RG with an STM32L476RG. Further close-ups are in Appendix B.

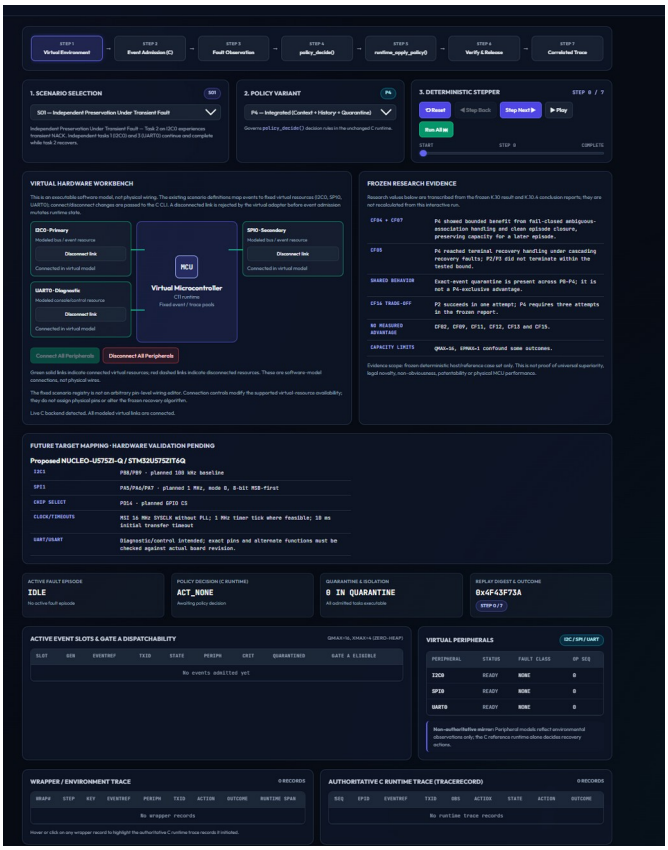


Fig. 7. *Virtual-system demonstrator. This is a software model and not a live connection to a microcontroller.*

8. Discussion

What the evidence supports. In the host campaign, a required exact association and a hard attempt limit remove two failure paths that the runtime exposes to the other variants: a leaked episode after a non-terminal action on an invalid target, and unbounded action sequences under repeated recovery faults. On the board, the same decision function and release path drove a real I²C peripheral through a reinitialization and a fresh verified read.

What it does not support. The CF04 and CF07 differences depend on a runtime defect that affects P0–P3, so a runtime that closed the episode would probably narrow them. This is a hypothesis; it was not tested, and the defect-corrected candidate's campaign outputs are not available to us. The baselines are our own and share quarantine with P4, so the comparison does not isolate quarantine and says nothing about independently designed policies. P2 is cheaper in CF16, and seven families are indistinguishable. A simpler policy suffices when faults are transient, known to be persistent or unambiguous.

Hardware and software. The software model and the board run the same decision source, and in the fixed hardware context the observed decisions match the host computation. That agreement is consistent with the model but does not validate the model for other faults. The control trials add that for this NACK the next read succeeds without reinitialization, so the hardware demonstrates that the recovery path can be driven and verified, not that it was required. Table 10 summarizes the claim boundaries.

Table 10. *Claims and their support.*

Claim	Status	Wording boundary
400 deterministic host executions across 16 families	Supported	Frozen table; regenerated in a scratch copy

Claim	Status	Wording boundary
P4 closes all 16 episodes	Supported	This model and these baselines only
P4 better than P0–P3	Too broad	Selected advantages in CF04, CF05, CF07
Quarantine is unique to P4	Rejected	Shared by P0–P4
P4 is more efficient	Rejected	CF16: 3 attempts against 1
Zero heap calls in the runtime source	Supported	Source search; no runtime measurement
Controlled NACK followed by verified fresh read on hardware	Supported	One setup; address NACK only
Five trials show P4 reliability	Rejected	Five control trials are not policy repetitions
80 physical fault injections	Rejected	80 decisions without bus access
Stuck-bus recovery	Not shown	Not tested
Live UI-to-MCU integration	Not established	Separate paths
Novelty or patentability of the combination	Not established	Unresolved

9. Threats to Validity and Limitations

Internal. The baselines were written from the same code base as P4, and the frozen record marks the ablation contract as not valid for P0–P3 because of shared quarantine. Outcome codes are not commensurable across families. The report's free-text findings disagree with its data columns for CF01 (P2 attempts) and CF08 (P0 outcome), and we used the data columns, which the regeneration confirmed. The original raw output files are missing from the repository.

Construct. Attempt counts are not time, and host decisions are not hardware behaviour. The 16 firmware contexts are provisional and not shown to equal CF01–CF16.

External. One MCU, one sensor, one bus and one fault type were used. The runtime was exercised only with at most 16 queued events and one episode.

Hardware evidence. The values for the P4 single run and the control trials come from the project's hardware records, with no separate original screenshot available; only the policy-comparison readings in Fig. 6 are supported by supplied captures. The comparison is one run per policy, so no repeatability, variance or statistical claim is possible. No complete 80-entry export exists, and sampled entries (indices 0, 15, 16, 31, 32, 47, 48, 63, 64 and 79) read as recorded. The flashed binary is not hash-linked to any committed source, and the supplied `hardware_exp.c` is not committed. The controlled fault is an address NACK and not a stuck bus, and injected faults from different techniques can differ in impact (Arlat et al., 2003). No heap use, timing, energy or CPU measurement was made on the MCU.

Implementation and literature. The runtime has the limits described in Section 4.5. The literature search was not systematic, patents were not reviewed, and full-text claim checks are incomplete for some references.

10. Conclusion and Future Work

We implemented a zero-heap recovery runtime with five policy variants and evaluated it in a deterministic host campaign of 400 executions and on an STM32L476RG with a BME280. The integrated policy P4 closed every episode in the 16 families, avoided two failure paths that simpler variants reach, was no better in seven families and was costlier than P2 in one. On hardware, a controlled address NACK was followed by policy-selected reinitialization and a fresh read of the expected chip ID for P1 and P4 in one comparison, for P4 in a separate run, and in five control trials, where a full reinitialization proved unnecessary for the next read. The software demonstrator and the board are separate evidence paths. Stuck-bus recovery, independent repeated policy trials, MCU resource measurements and general superiority remain unestablished.

Next steps are: repeated controlled trials with a no-reinitialization control for each policy; a safely designed stuck-SDA or stuck-SCL test; MCU measurement of Flash, RAM, stack and decision time; independently designed baselines; reporting the defect-corrected runtime with its own campaign; and a systematic literature and patent review.

Data and code availability

The code, reports and derived data are in the public repository SujitSaiY2007/embedded-runtime-resilience (commit 982dad1 at the time of writing). The original raw K.10 output files and the original hardware screenshots for the P4 single run and the control trials are not in the repository.

CRediT authorship contribution statement

Sujit Sai Y: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft. **Mithul Rashwanth SR:** Validation, Investigation, Writing – review & editing. **Rohit Mathur:** Supervision, Project administration, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics statement

Ethics approval was not applicable. The work involved embedded firmware, software experiments and a sensor-based hardware demonstration, with no human participants and no personal data.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used ChatGPT (OpenAI) and Claude (Anthropic) to audit repository records, draft and edit manuscript text, write the scripts that draw the explanatory diagrams and the host-outcome figure, and run verification builds and a consistency regeneration of the host campaign in a scratch environment. The tools were not used to create experimental images or measurements: the photograph, debugger captures and demonstrator screenshots are authentic project captures. The authors reviewed and edited the content and take full responsibility for the content of the publication. AI tools are not authors.

References

Albalooshi, A., Jallad, A. H. M., & Marpu, P. R. (2023). Fault analysis and mitigation techniques of the I2C bus for nanosatellite missions. IEEE Access, 11, 34709–34717. https://doi.org/10.1109/ACCESS.2023.3262410

Arlat, J., Crouzet, Y., Karlsson, J., Folkesson, P., Fuchs, E., & Leber, G. H. (2003). Comparison of physical and software-implemented fault injection techniques. IEEE Transactions on Computers, 52(9), 1115–1133. https://doi.org/10.1109/TC.2003.1228509

Asghari, S. A., Binesh Marvasti, M., & Rahmani, A. M. (2018). Enhancing transient fault tolerance in embedded systems through an OS task level redundancy approach. Future Generation Computer Systems, 87, 58–65. https://doi.org/10.1016/j.future.2018.04.049

Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. IEEE Transactions on Dependable and Secure Computing, 1(1), 11–33. https://doi.org/10.1109/TDSC.2004.2

Branco, A., Mottola, L., Alizai, M. H., & Siddiqui, J. H. (2019). Intermittent asynchronous peripheral operations. Proceedings of the 17th ACM Conference on Embedded Networked Sensor Systems (SenSys '19), 55–67. https://doi.org/10.1145/3356250.3360033

Candea, G., Kawamoto, S., Fujiki, Y., Friedman, G., & Fox, A. (2004). Microreboot—A technique for cheap recovery. Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04), 31–44. USENIX Association.

Carreira, J., Madeira, H., & Silva, J. G. (1998). Xception: A technique for the experimental evaluation of dependability in modern computers. IEEE Transactions on Software Engineering, 24(2), 125–136.

Heffernan, D., MacNamee, C., & Fogarty, P. (2014). Runtime verification monitoring for automotive embedded systems using the ISO 26262 functional safety standard as a guide for the definition of the monitored properties. IET Software, 8(5), 193–203. https://doi.org/10.1049/iet-sen.2013.0236

Kempa, B., Zhang, P., Jones, P. H., Zambreno, J., & Rozier, K. Y. (2020). Embedding online runtime verification for fault disambiguation on Robonaut2. Formal Modeling and Analysis of Timed Systems (FORMATS 2020), LNCS 12288, 196–214. Springer. https://doi.org/10.1007/978-3-030-57628-8_12

Kleeberger, P. R., Rivera, J., Mueller-Gritschneder, D., & Schlichtmann, U. (2019). SeRoHAL: Generation of selectively robust hardware abstraction layers for efficient protection of mixed-criticality systems. 24th Asia and South Pacific Design Automation Conference (ASP-DAC 2019). https://doi.org/10.1145/3287624.3287652

Natella, R., Cotroneo, D., & Madeira, H. S. (2016). Assessing dependability with software fault injection: A survey. ACM Computing Surveys, 48(3). https://doi.org/10.1145/2841425

Reinbacher, T., Függer, M., & Brauer, J. (2014). Runtime verification of embedded real-time systems. Formal Methods in System Design, 44, 203–239. https://doi.org/10.1007/s10703-013-0199-z

Rodriguez Arreola, A., Balsamo, D., Merrett, G. V., & Weddell, A. S. (2018). RESTOP: Retaining external peripheral state in intermittently-powered sensor systems. Sensors, 18(1), 172. https://doi.org/10.3390/s18010172

Safari, S., Ansari, M., Khdr, H., Gohari Nazari, P., Yari-Karin, S., Yeganeh-Khaksar, A., Hessabi, S., Ejlali, A., & Henkel, J. (2022). A survey of fault-tolerance techniques for embedded systems from the perspective of power, energy, and thermal issues. IEEE Access, 10, 12229–12251. https://doi.org/10.1109/ACCESS.2022.3144217

Santos, R. M., Santos, J., & Orozco, J. D. (2009). Power saving and fault-tolerance in real-time critical embedded systems. Journal of Systems Architecture, 55(2), 90–101. https://doi.org/10.1016/j.sysarc.2008.09.001

Smith, R., & Rixner, S. (2015). Surviving peripheral failures in embedded systems. Proceedings of the 2015 USENIX Annual Technical Conference (USENIX ATC 15). USENIX Association. https://www.usenix.org/conference/atc15/technical-session/presentation/smith

Solouki, M. A., Angizi, S., & Violante, M. (2024). Dependability in embedded systems: A survey of fault tolerance methods and software-based mitigation techniques. IEEE Access, 12, 180939–180967. https://doi.org/10.1109/ACCESS.2024.3509633

Swift, M. M., Annamalai, M., Bershad, B. N., & Levy, H. M. (2006). Recovering device drivers. ACM Transactions on Computer Systems, 24(4), 333–360. https://doi.org/10.1145/1189256.1189257

Thati, V. B., Vankeirsbilck, J., Boydens, J., & Pissoort, D. (2017). Data error detection and recovery in embedded systems: A literature review. Advances in Science, Technology and Engineering Systems Journal, 2(3), 623–633. https://doi.org/10.25046/aj020380

Trivedi, K. S., Grottke, M., & Lopez, J. A. (2024). Rethinking software fault tolerance. IEEE Transactions on Reliability, 73(1), 67–72. https://doi.org/10.1109/TR.2023.3330787

Appendix A. Artifact Integrity

Table 11. *Artifacts and recorded hashes.*

Artifact	Record
`reference/src/ref_runtime.c`	SHA-256 c19e812e274f0fca8a718d051099c34eb36ac08cc4c432e78db9441a4ffecf37
`reference/include/ref_runtime.h`	SHA-256 4cf5fa3c91032bc4395764d16ed51d9c5462e298001e42d7f6f9644e5b41be35
Hardware source supplied by the authors (`hardware_exp.c`)	SHA-256 8d65294a7dbf8aca22cab9f2c56732bdb015bf7c3fab87cee6b555f7ae6a1ee8; identifies the source bytes, not the flashed binary
Frozen host report	`research/phase2_k10_p4_advantage_novelty_results.md` (data columns used)
Hardware records	`Plan2.0/results/` (debugger values recorded by the authors)

Appendix B. Supplementary Demonstrator Captures

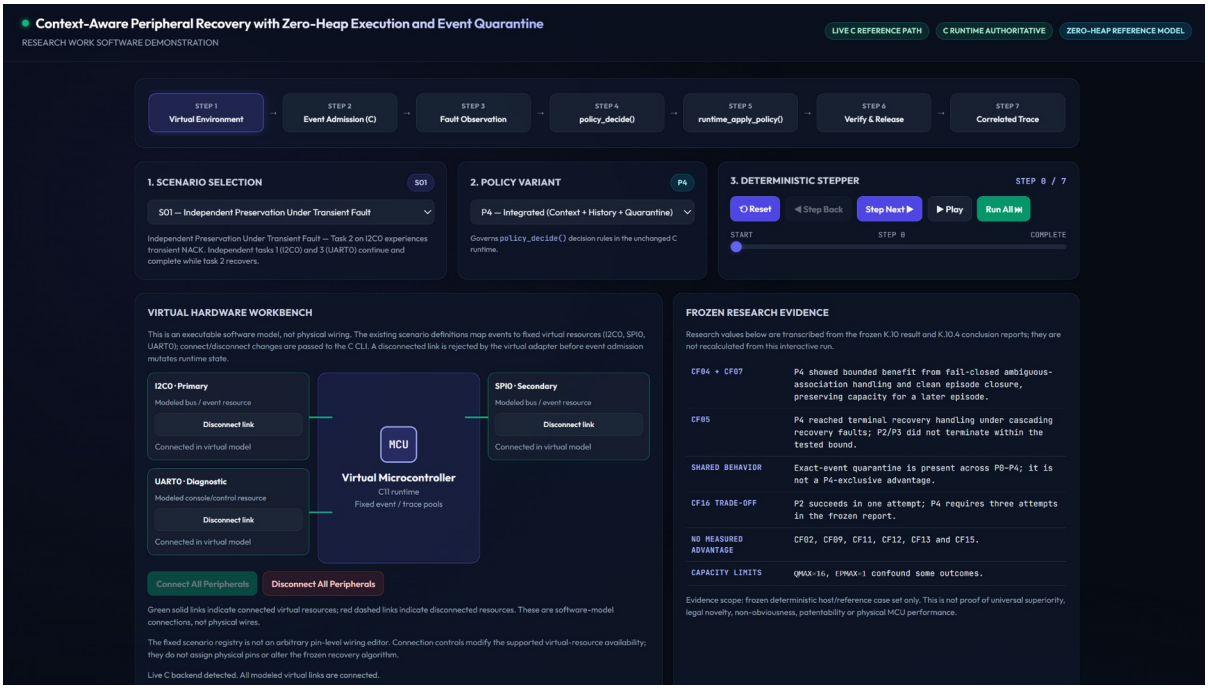


Fig. S1. *Virtual hardware workbench and frozen research evidence panel. Values should be cross-checked against the frozen report.*

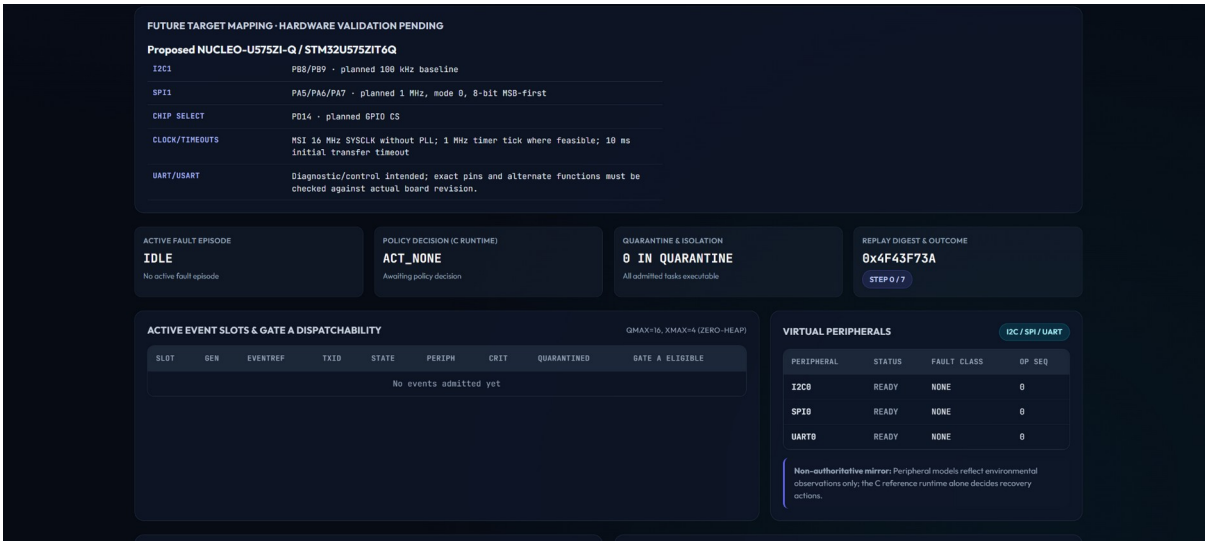


Fig. S2. *Runtime status, virtual peripherals and event tables at the initial step. Empty trace tables mean no events had been admitted.*

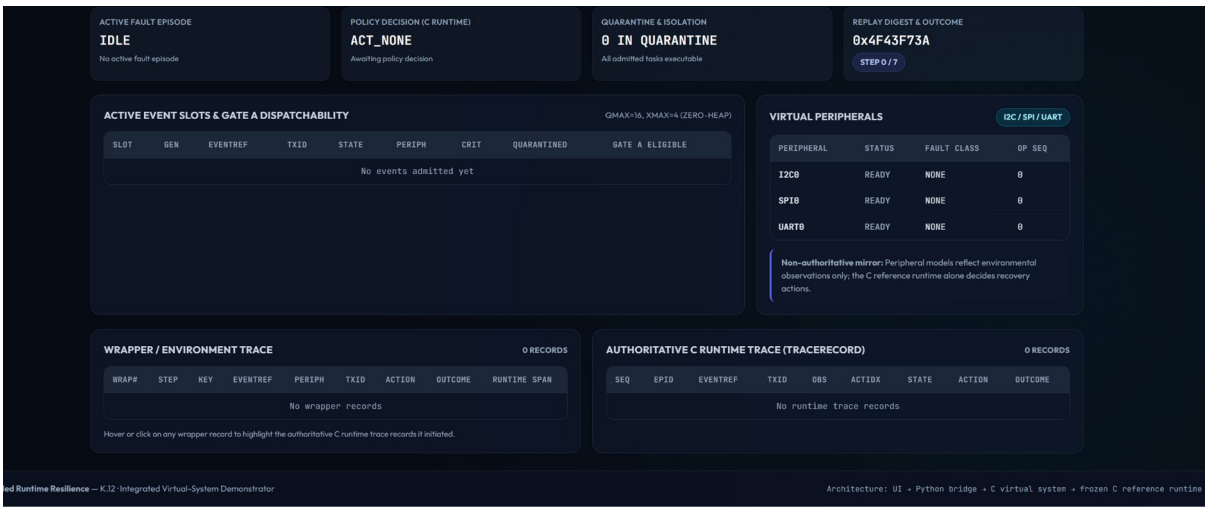


Fig. S3. *Peripheral status and trace panels at the initial step. This is interface documentation and not evidence of a recovery event.*